

0. All about Performance Testing in details

- [Overview](#)
- [Objectives of Performance Testing](#)
- [Types of Performance Testing](#)
- [Performance Testing Approach](#)
 - [1. How We Test](#)
 - [2. Where We Test](#)
 - [3. When to Test](#)
- [Tools & Frameworks](#)
- [Execution & Reporting](#)
 - [1. Execution Process](#)
 - [2. Reporting Guidelines](#)
- [Best Practices](#)
- [Final Notes](#)
 - [References](#)

Overview

Performance testing is critical to ensure that our applications meet **scalability, reliability, and responsiveness** expectations. This strategy outlines the **types of performance tests, tools, environments, execution process, and reporting guidelines** to optimize application performance.

Objectives of Performance Testing

1. **Ensure SLA compliance** – Verify that response times are within acceptable thresholds (e.g., 3 seconds per page load).
2. **Validate system stability under load** – Ensure application performance remains consistent during peak usage.
3. **Detect performance degradation over time** – Identify memory leaks, CPU spikes, and performance bottlenecks.
4. **Assess scalability** – Evaluate how the application behaves when user load increases by 50% or more.
5. **Identify failure thresholds** – Determine system limits through stress and spike testing.

Types of Performance Testing

Performance testing encompasses various methodologies to evaluate a system's responsiveness, stability, and scalability under different conditions.

Below is a concise overview of each performance testing type, detailing their purposes, key considerations, and recommendations on their necessity.

Test Type	Purpose	Key Considerations	Recommendation
Load Testing	Assess system behavior under expected user loads to ensure it meets performance criteria.	Monitor response times, throughput, and resource utilization. Identify bottlenecks that could affect user experience.	Essential for all critical applications to validate performance under anticipated conditions.
Stress Testing	Determine system robustness by testing beyond normal operational capacity, identifying the breaking point.	Gradually increase load until the system fails. Observe how and when failures occur.	Important for applications where understanding limits is crucial; can be scheduled periodically.
Soak (Endurance) Testing	Evaluate system performance over an extended period to detect issues like memory leaks or degradation.	Run tests over prolonged durations at normal load levels. Monitor for performance degradation over time.	Beneficial for applications requiring long-term reliability; conduct based on system criticality.
Spike Testing	Analyze system response to sudden, extreme increases in load to assess stability and recovery.	Introduce abrupt load spikes and monitor system's ability to handle and recover.	Situational ; perform when applications are susceptible to sudden traffic surges.
Scalability Testing	Measure system's ability to scale up or down in response to load changes, ensuring it meets performance expectations.	Adjust system resources and observe performance variations. Identify scaling limits and inefficiencies.	Advisable when planning for growth or cloud-based scaling; schedule as needed.
Volume Testing	Examine system performance with large volumes of data to ensure data handling capabilities.	Input significant data volumes and monitor processing times and system behavior.	Optional ; useful for data-intensive applications but can be deprioritized if not applicable.

Configuration Testing	Assess system performance under various configuration settings to identify optimal setups.	Modify system configurations and evaluate performance impacts.	Optional ; conduct during initial setup or major changes.
Isolation Testing	Identify specific system components causing performance issues by isolating and testing them individually.	Test components separately to pinpoint performance bottlenecks.	Optional ; utilize when troubleshooting specific issues.

Key Recommendations:

- **Essential Tests:** Load Testing is fundamental for all critical applications to ensure they can handle expected user loads effectively.
- **Important Tests:** Stress Testing and Soak Testing are valuable for understanding system limits and long-term reliability. Schedule these based on application criticality and usage patterns.
- **Situational Tests:** Spike Testing and Scalability Testing should be conducted when specific conditions warrant, such as anticipated traffic surges or planned scaling.
- **Optional Tests:** Volume, Configuration, and Isolation Testing can be performed as needed, depending on the application's nature and any arising performance concerns.

By strategically implementing these performance testing types, organizations can ensure their applications deliver optimal performance, reliability, and user satisfaction.

Reference: More details on test types – [Blazemeter Blog](#)

Performance Testing Approach

1. How We Test

We perform performance tests at both **API** and **UI** levels using **Blazemeter**, **Taurus**, and **JMeter**.

- **API Testing:** Validate endpoint performance with **Postman**, **REST Assured**.
- **UI Testing:** Simulate real-world user interactions using **Blazemeter/Taurus**.
- **Batch Processing:** Measure execution time for batch jobs (e.g., database operations).

Note: UI performance testing via **Selenium/Playwright** is under evaluation due to resource overhead.

2. Where We Test

Environment	Usage
Production-Like (Preferred)	Best for realistic performance results
QA/UAT Environments	Useful for script validation & regression tests
Non-Production (Dev, Staging)	Ideal for early-stage testing, not for benchmarking

Production Testing Considerations:

- Secure **CAB approvals** before execution.
- Perform tests **during off-peak hours**.
- Involve **DBA & DevOps** to monitor system health.

3. When to Test

- New application or service launch.
- Major code changes affecting performance.
- Business growth projections requiring higher capacity.

Tools & Frameworks

Category	Tools
Load Testing	Blazemeter, JMeter, Taurus
API Testing	Postman, REST Assured
UI Testing	Blazemeter/Taurus (Recommended), <i>Selenium/Playwright (Under Review)</i>

Monitoring	Splunk (<i>Substitute: New Relic</i>)
CI/CD Integration	Jenkins, Azure DevOps

Execution & Reporting

1. Execution Process

1. **Define Test Scenarios** – Identify key workflows (login, transactions, API calls).
2. **Set Load Parameters** – Specify concurrent users, duration, ramp-up strategy.
3. **Run Tests** – Execute scripts in **Blazemeter/Taurus/JMeter**.
4. **Monitor in Real-Time** – Track response times, error rates, and resource utilization.
5. **Analyze & Report** – Document findings and share insights with stakeholders.

2. Reporting Guidelines

- Use **Blazemeter Reports** for graphical analysis.
 - Capture **response time trends, error percentages, throughput**.
 - Document results in **Confluence pages** with screenshots and **report links**.
-

Best Practices

Always baseline before running complex tests
Use production-like environments for accurate results
Monitor CPU, Memory, and Database performance alongside application metrics
Integrate tests into CI/CD to detect regressions early
Scale incrementally—adjust resources and retest for optimization

Final Notes

Performance testing is a **continuous process** and should be incorporated into every release cycle. The goal is to **ensure optimal user experience, prevent bottlenecks, and maintain system reliability**.

Next Steps: Implement a **standardized performance testing framework** across teams, ensuring **automated regression performance tests** are included in every release.

References

- **Blazemeter Docs:** [Performance Testing Overview](#)
- **Taurus Documentation:** [Blazemeter with Taurus](#)
- **CI/CD Integration Guide:** [Jenkins & Blazemeter](#)